# Programming in C
# Unit 3
# Functions

# C Functions

- A function is a group of statements that together perform a task.
- There are two different types of Functions:
  - In built functions : The C standard library provides numerous built-in functions. For example, strcat( ), printf( ), scanf( ) etc.
  - User defined functions : The user defined functions are created by the programmer for its use.

# C Functions

- A function has following three aspects:
- A function **declaration** tells the compiler about a function's name, return type, and parameters.
- A function **definition** provides the actual body of the function.
- **Function call** can be done from anywhere in the program.

# C Functions

- A function may or may not accept any argument. It may or may not return any value. Therefore, there are four different aspects of function calls.
- function without arguments and without return value
- function without arguments and with return value
- function with arguments and without return value
- function with arguments and with return value

# Function without arguments and without return value

**Example:**

```
#include<stdio.h>
void print();   //function declaration
int main()
{
   printf("Hii!");  // function call
   print();
}
void print()    // function definition
{
   printf("\nThis is a class");
}
```

# Function without arguments and with return value

**Example:**

```
#include<stdio.h>
int sum();   // Function declaration
void main()
{
   int result;
   printf("\nGoing to calculate the sum of two numbers:");
   result = sum();  // function call
   printf("%d",result);
}
int sum()  // function definition
{
   int a,b;
   printf("\nEnter two numbers");
   scanf("%d %d",&a,&b);
   return a+b;
}
```

# Function with arguments and without return value

**Example:**

```
#include<stdio.h>
void sum(int, int);
void main()
{
   int a,b,result;
   printf("\nGoing to calculate the sum of two numbers:");
   printf("\nEnter two numbers:");
   scanf("%d %d",&a,&b);
   sum(a,b);
}
void sum(int a, int b)
{
   printf("\nThe sum is %d",a+b);
}
```

# Function with arguments and with return value

**Example:**

```
#include<stdio.h>
int even_odd(int);
void main()
{    int n,flag=0;
     printf("\n To check whether a number is even or odd");
     printf("\nEnter the number: ");
     scanf("%d",&n);
     flag = even_odd(n);
     if(flag == 0)
          {  printf("\nThe number is odd");   }
     else
          {  printf("\nThe number is even");   }
}
int even_odd(int n)
{  if(n%2 == 0)
   {  return 1;   }
   else
   {  return 0;   }
}
```

# Local variables and Global Variables

- There are three places where variables can be declared in C programming language −
  - Inside a function or a block which is called local variables.
  - Outside of all functions which are called global variables.
  - In the definition of function parameters which are called formal parameters.

# Local and Global Variables in C Language

- Variables that are declared inside a function or block are called local variables. They can be used only by statements that are inside that function or block of code. Local variables are not known to functions outside their own.
- Global variables are defined outside a function, usually on top of the program. Global variables hold their values throughout the lifetime of the program and they can be accessed inside any of the functions defined for the program.

# Example to Understand Global and local Variables in C Language

```c
#include <stdio.h>
/* global variable declaration */
int g;
int main ()
{
    /* local variable declaration */
    int a, b;
    /* actual initialization */
    a = 10;
    b = 20;
    g = a + b;
    printf ("value of a = %d, b = %d and g = %d\n", a, b, g);
    return 0;
}
```

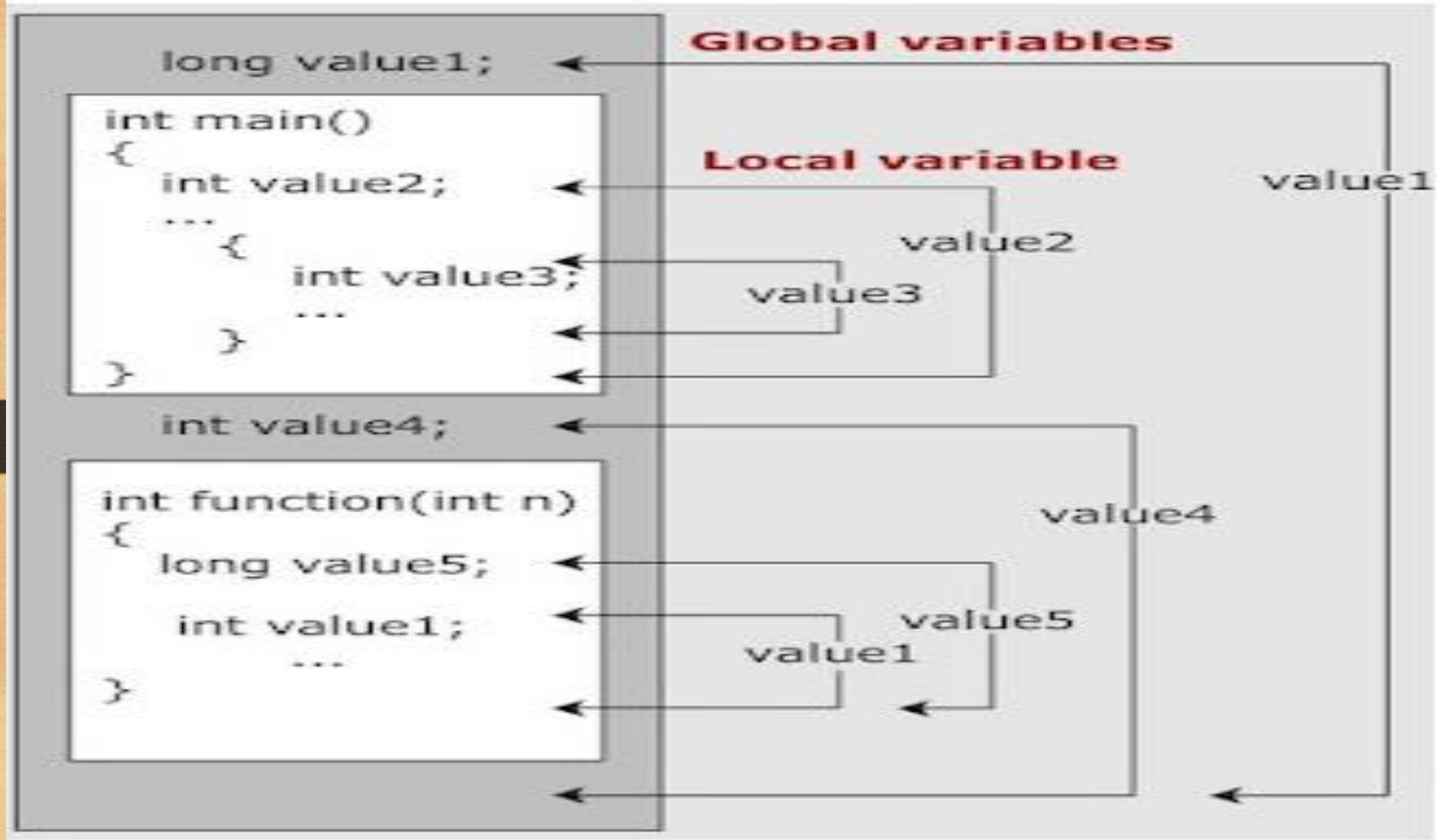
Output: `value of a = 10, b = 20 and g = 30`

## Local and Global Variables with the same name in C Language:

A program can have the same name for local and global variables but the value of the local variables inside a function will take preference.

```c
#include <stdio.h>
/* global variable declaration */
int g = 20;
int main ()
{
    /* local variable declaration */
    int g = 10;
    printf ("value of g = %d\n", g);
    return 0;
}
```

Output: `value of g = 10`


Global variables
long value1;
int main()
{
int value2;
...
{
int value3;
...
}
}
Local variable
value1
value2
value3
int value4;
int function(int n)
{
long value5;
int value1;
...
}
value4
value5
value1

# Formal Parameters of a Function in C Language

- The Formal Parameters in C Programming Language are treated as local variables within a function and they take precedence over the global variables if any.

```c
//Program to Understand Formal Parameters of a Function in C Language
#include <stdio.h>
int a = 20;      /* global variable declaration */
int sum(int a, int b);
int main ()
{
    /* local variable declaration in main function */
    int a = 10;
    int b = 20;
    int c = 0;
    printf ("value of a in main() = %d\n",  a);
    c = sum( a, b);
    printf ("value of c in main() = %d\n",  c);
    return 0;
}
/* function to add two integers */
int sum(int a, int b)
{
     printf ("value of a in sum() = %d\n",  a);
     printf ("value of b in sum() = %d\n",  b);
     return a + b;
}
```

```
C:\Program Files (x86)\Dev-Cpp\ConsolePauser.exe
value of a in main() = 10
value of a in sum() = 10
value of b in sum() = 20
value of c in main() = 30

--------------------------------
Process exited with return value 0
Press any key to continue . . .
```

# Call by value and call be reference

- Functions can be invoked in two ways: Call by Value or Call by Reference. These two ways are generally differentiated by the type of values passed to them as parameters.

- In function header or in function declarator whatever the variables we are creating those are called parameters or **formal arguments**. On the other hand, in function calling statement whatever the data we are passing those are called **actual arguments** or arguments.

# Call by Value

- In call by value method of parameter passing, the values of actual parameters are copied to the function's formal parameters.
  - There are two copies of parameters stored in different memory locations.
  - One is the original copy and the other is the function copy.
  - Any changes made inside functions are not reflected in the actual parameters of the caller.

```c
//Program to illustrate call by value
#include <stdio.h>
void swap(int , int); //prototype of the function
int main()
{
    int a = 10;
    int b = 20;
    printf("Before swapping the values in main a = %d, b = %d\n",a,b); // printing the value of a and b in main
    swap(a,b);
    printf("After swapping values in main a = %d, b = %d\n",a,b); // The value of actual parameters do not change by
    //changing the formal parameters in call by value, a = 10, b = 20
}
void swap (int a, int b)
{
    int temp;
    temp = a;
    a=b;
    b=temp;
    printf("After swapping values in function a = %d, b = %d\n",a,b); // Formal parameters, a = 20, b = 10
}
```

```
C:\Program Files (x86)\Dev-Cpp\ConsolePauser.exe
Before swapping the values in main a = 10, b = 20
After swapping values in function a = 20, b = 10
After swapping values in main a = 10, b = 20
```

# Call by Reference

- In call by reference method of parameter passing, the address of the actual parameters is passed to the function as the formal parameters.
  - Both the actual and formal parameters refer to the same locations.
  - Any changes made inside the function are actually reflected in the actual parameters of the caller.

```c
//Program to illustrate call by reference
#include <stdio.h>
void swap(int *, int *); //prototype of the function
int main()
{
int a = 10;
int b = 20;
printf("Before swapping the values in main a = %d, b = %d\n",a,b); // printing the value of a and b in main
swap(&a,&b);
printf("After swapping values in main a = %d, b = %d\n",a,b); // The values of actual parameters do change in call by reference
//a = 10, b = 20
}
void swap (int *a, int *b)
{
int temp;
temp = *a;
*a=*b;
*b=temp;
printf("After swapping values in function a = %d, b = %d\n",*a,*b); // Formal parameters, a = 20, b = 10
}
```

```
C:\Program Files (x86)\Dev-Cpp\ConsolePauser.exe
Before swapping the values in main a = 10, b = 20
After swapping values in function a = 20, b = 10
After swapping values in main a = 20, b = 10


--------------------------------
```

| Call By Value | Call By Reference |
| --- | --- |
| While calling a function, we pass the values of variables to it. Such functions are known as "Call By Values". | While calling a function, instead of passing the values of variables, we pass the address of variables(location of variables) to the function known as "Call By References. |
| In this method, the value of each variable in the calling function is copied into corresponding dummy variables of the called function. | In this method, the address of actual variables in the calling function is copied into the dummy variables of the called function. |
| With this method, the changes made to the dummy variables in the called function have no effect on the values of actual variables in the calling function. | With this method, using addresses we would have access to the actual variables and hence we would be able to manipulate them. |
| In call-by-values, we cannot alter the values of actual variables through function calls. | In call by reference, we can alter the values of variables through function calls. |
| Values of variables are passed by the Simple technique. | Pointer variables are necessary to define to store the address values of variables. |

# Recursion

- The C programming language allows any of its functions to call itself multiple times in a program. Here, any function that happens to call itself again and again (directly or indirectly), unless the program satisfies some specific condition/subtask is called a recursive function.

**For Example:**

```
int func( )
{
    .....
    func( );
}
```

- While using recursion, programmers need to be careful to define an exit condition from the function, otherwise it will go into an infinite loop.

# Demonstration of recursion

```
int fun(int n )
{
    if  (n==1)
        return 1;
    else
        return 1+ fun(n-1);
}
```

```
int main( )
{
    n=3;
    printf("%d", fun(n));
    return 0;
}
```

# Output of the following Program?

```
#include<stdio.h>
int fun(int n )
{
   if (n==0)
        return 1;
   else
         return 5+ fun(n-2);
}
```

```
int main( )
{
   printf("%d", fun(4));
   return 0;
}
```

# Advantages and Disadvantages of Recursion in C

- **Advantages:**
- Recursion is more elegant and requires a lesser number of variables which makes the program short and clean.
- Recursion can be made to replace complex nesting codes since we don't have to call the program, again and again, to do the same task as it calls itself.
- **Disadvantages:**
- It is slower than non recursive programs due to the overhead of maintaining the stack.
- It requires more memory for the stack.
- For better performance, use loops instead of recursion. Because recursion is slower.

# Practice question

```
#include <stdio.h>
int fact (int);
int main ()
{
   int n, result;
   printf  ("Enter a number whose factorial is to be calculated: ");
   scanf  ("%d", &n);
   if  (n < 0)
      { printf  ("Factorial does not exist."); }
      else
      { result = fact(n);
       printf("Factorial of %d is %d.", n,result); }
   return 0; }
//recursion function definition
 int fact (int n)
{ if  (n == 0 || n == 1)
 return 1;
else return (n * fact (n - 1)); //calling function definition }
```

# Storage Classes

- C Storage Classes are used to describe the features of a variable/function. These features basically include the scope, visibility, and lifetime which help us to trace the existence of a particular variable during the runtime of a program.

| Storage Specifier | Initial value | Scope | Life |
|---|---|---|---|
| auto | Garbage | Within block | End of block |
| extern | Zero | global Multiple files | Till end of program |
| static | Zero | Within block | Till end of program |
| register | Garbage | Within block | End of block |

# Auto

- This is the default storage class for all the variables declared inside a function or a block. Hence, the keyword auto is rarely used while writing programs in C language. Auto variables can be only accessed within the block/function they have been declared and not outside them (which defines their scope). Of course, these can be accessed within nested blocks within the parent block/function in which the auto variable was declared.
- However, they can be accessed outside their scope as well using the concept of pointers given here by pointing to the very exact memory location where the variables reside. They are assigned a garbage value by default whenever they are declared.

# Extern

- Extern storage class simply tells us that the variable is defined elsewhere and not within the same block where it is used. Basically, the value is assigned to it in a different block and this can be overwritten/changed in a different block as well. So an extern variable is nothing but a global variable initialized with a legal value where it is declared in order to be used elsewhere. It can be accessed within any function/block.
- Also, a normal global variable can be made extern as well by placing the ‘extern’ keyword before its declaration/definition in any function/block. This basically signifies that we are not initializing a new variable but instead, we are using/accessing the global variable only. The main purpose of using extern variables is that they can be accessed between two different files which are part of a large program.

# static

- This storage class is used to declare static variables which are popularly used while writing programs in C language. Static variables have the property of preserving their value even after they are out of their scope! Hence, static variables preserve the value of their last use in their scope. So we can say that they are initialized only once and exist till the termination of the program. Thus, no new memory is allocated because they are not re-declared.
- Their scope is local to the function to which they were defined. Global static variables can be accessed anywhere in the program. By default, they are assigned the value 0 by the compiler.

# register

- This storage class declares register variables that have the same functionality as that of the auto variables. The only difference is that the compiler tries to store these variables in the register of the microprocessor if a free register is available. This makes the use of register variables to be much faster than that of the variables stored in the memory during the runtime of the program.
- If a free registration is not available, these are then stored in the memory only. Usually, a few variables which are to be accessed very frequently in a program are declared with the register keyword which improves the running time of the program. An important and interesting point to be noted here is that we cannot obtain the address of a register variable using pointers.

```c
// A C program to demonstrate different storage classes
#include <stdio.h>
int x; // declaring the variable which is to be made extern an initial value can also be initialized to x
void autoStorageClass()
{   printf("\nDemonstrating auto class");
     auto int a = 32; // writing "int a=32;" works as well)
     printf("Value of the variable 'a' declared as auto: %d\n", a);
    printf("--------------------------------"); }
void registerStorageClass()
{   printf("\nDemonstrating register class\n\n");
    register char b = 'G';
    printf("Value of the variable 'b' declared as register: %d\n",b);
    printf("--------------------------------"); }
void externStorageClass()
{   printf("\nDemonstrating extern class\n\n");
extern int x; // telling compiler that the variable,x is an extern variable and has beendefined elsewhere (above the main)
    printf("Value of the variable 'x' declared as extern: %d\n", x);
    x = 2;  // value of extern variable x modified
   printf("Modified value of the variable 'x' declared as extern: %d\n", x);
   printf("--------------------------------"); }
```

```c
void staticStorageClass()
{  int i = 0;
  printf("\nDemonstrating static class\n\n");
  printf("Declaring 'y' as static inside the loop.\n But this declaration will occur only once as 'y' is static.\n"
          "If not, then every time the value of 'y' will be the declared value as in the case of variable 'p'\n");
   printf("\nLoop started:\n");
    for (i = 1; i < 5; i++) {
          static int y = 5;
          int p = 10;
          y++;
          p++;
        printf("\nThe value of 'y',declared as static, in %d " "iteration is %d\n", i, y);
        printf("The value of non-static variable 'p', " "in %d iteration is %d\n", i, p); }
      printf("\nLoop ended:\n");
      printf("--------------------------------"); }
int main()
{  printf("A program to demonstrate Storage Classes in C\n\n");
    autoStorageClass();
    registerStorageClass();
    externStorageClass();
    staticStorageClass();
    printf("\n\nStorage Classes demonstrated");
    return 0; }
```

```
A program to demonstrate Storage Classes in C


Demonstrating auto classValue of the variable 'a' declared as auto: 32
------------------------------
Demonstrating register class

Value of the variable 'b' declared as register: 71
------------------------------
Demonstrating extern class

Value of the variable 'x' declared as extern: 0
Modified value of the variable 'x' declared as extern: 2
------------------------------
Demonstrating static class

Declaring 'y' as static inside the loop.
 But this declaration will occur only once as 'y' is static.
If not, then every time the value of 'y' will be the declared value as in the case of variable 'p'

Loop started:

The value of 'y',declared as static, in 1 iteration is 6
The value of non-static variable 'p', in 1 iteration is 11

The value of 'y',declared as static, in 2 iteration is 7
The value of non-static variable 'p', in 2 iteration is 11

The value of 'y',declared as static, in 3 iteration is 8
The value of non-static variable 'p', in 3 iteration is 11

The value of 'y',declared as static, in 4 iteration is 9
The value of non-static variable 'p', in 4 iteration is 11

Loop ended:
------------------------------

Storage Classes demonstrated
------------------------------
```